

CYBER-PHYSICAL SMART BUILDING ACCESS CONTROL SYSTEM

Final Graduation Project Report

Submitted in fulfillment of the graduation project requirement

Abstract

This report presents the design, implementation, and testing of a cyber-physical access control system for a multi-building academic environment. The system combines credential-based physical entry (RFID/NFC with optional biometric second factor), cryptographic card authentication, tamper detection with an active fail-secure response, a hybrid RS-485/Wi-Fi communication backbone supporting multiple door nodes per building, a FastAPI backend with encrypted credential storage and role-based access control, and a live web dashboard for monitoring and remote override.

The project was carried out in ten phases spanning proposal and planning through system design, a single-node hardware prototype, backend and database implementation, dashboard development, security hardening, multi-node scaling, full-system testing, documentation, and defense preparation. Wherever feasible, deliverables were built and executed for real rather than only described: the backend was run as a live server and exercised with real HTTP/MQTT traffic (43 automated tests plus live integration runs), the dashboard was built and integration-tested against that live backend, a from-scratch RS-485 wire protocol was verified byte-for-byte across two independent language implementations, and a full-system fault-injection test uncovered and fixed a real integration defect. The one component that could not be verified against real hardware in this project's development environment — the ESP32 firmware — is documented explicitly as reviewed-but-unverified throughout, rather than presented as more complete than it is.

Table of Contents

1. Introduction 2. Related Work 3. Requirements 4. System Architecture 5. Implementation 6. Security Design 7. Testing & Verification 8. Results & Evaluation 9. Project Management 10. Conclusion & Future Work 11. References Appendices

1. Introduction

1.1 Problem Statement

Physical access control in academic and institutional buildings is often handled by mechanical keys or standalone card readers with no central audit trail, no way to remotely respond to an incident, and no cryptographic protection against a cloned credential. A lost key means re-keying a lock; a lost or cloned card is often not detected until after it has been misused. Tamper events (forced entry, a reader physically attacked) are frequently not detected in real time at all.

1.2 Objectives

- Design and implement a door-level access control node combining RFID/NFC card authentication, an optional biometric second factor, and cryptographic (not just UID) verification of card identity.
- Provide a central backend that ingests live status/events from every door, encrypts credential data at rest, enforces role-based access control, and exposes both a REST API and a live dashboard.
- Support scaling from a single door to multiple doors per building over a hybrid wired (RS-485) / wireless (Wi-Fi) backbone, with automatic fallback.
- Treat security as a first-class requirement rather than an afterthought: encrypted storage, brute-force protection, tamper response, and an explicit threat model and gap analysis.
- Verify claims with real, executed tests wherever the project's environment allows, and state plainly where it doesn't (specifically: real ESP32 hardware and a real RS-485 electrical bus were not available during development).

1.3 Scope

The delivered system covers the full stack from door hardware firmware through a multi-node communication backbone to a backend and dashboard, sized for a pilot deployment (proposal budget: \$635 for a 5-door pilot) with a clear, documented path to a larger building-wide rollout (PostgreSQL instead of SQLite, per-card key diversification, real hardware bench-testing).

2. Related Work

Commercial access control systems (e.g. HID, Lenel, Genetec) provide mature, certified hardware but are closed platforms with licensing costs that scale poorly for a pilot deployment and limited ability to customize the firmware-level security model. Open-source home-automation access solutions (e.g. ESPHome-based RFID locks) are inexpensive and flexible but generally stop at UID-only card matching, without cryptographic card authentication, tamper-active response, or a hybrid wired/wireless backbone designed for multi-door scaling. This project sits between those two: built on inexpensive, widely available hardware (ESP32, PN532, MAX485) like the hobbyist solutions, but with the cryptographic authentication (MIFARE DESFire EV1 native AES), encrypted-at-rest credential storage, and structured multi-node architecture more typical of commercial systems.

3. Requirements

Category	Requirement
Functional	Credential-based unlock (card, optional fingerprint); remote admin override; scheduled/automatic unlock windows; live dashboard status
Functional	Central audit log of every access attempt (granted/denied) and every admin override
Security	Cryptographic card authentication (not UID-only); encrypted credential storage; role-based access control; login brute-force protection
Security	Tamper detection with an active response (force relock + lockout), not just a passive alert
Safety	Fail-secure or fail-safe lock behavior per door; REX button and mechanical key override independent of firmware/network state
Scalability	Support multiple doors per building over a shared communication backbone with a documented fallback path

Category	Requirement
Non-functional	Low-cost pilot hardware; zero-setup local development (SQLite) with a documented upgrade path (PostgreSQL) for production scale

4. System Architecture

The system is organized into three layers, matching the original System Design Document: an Edge Layer (door hardware and firmware), a Network Layer (the hybrid RS-485/Wi-Fi backbone and the Building Gateway), and an Application Layer (the backend and dashboard).

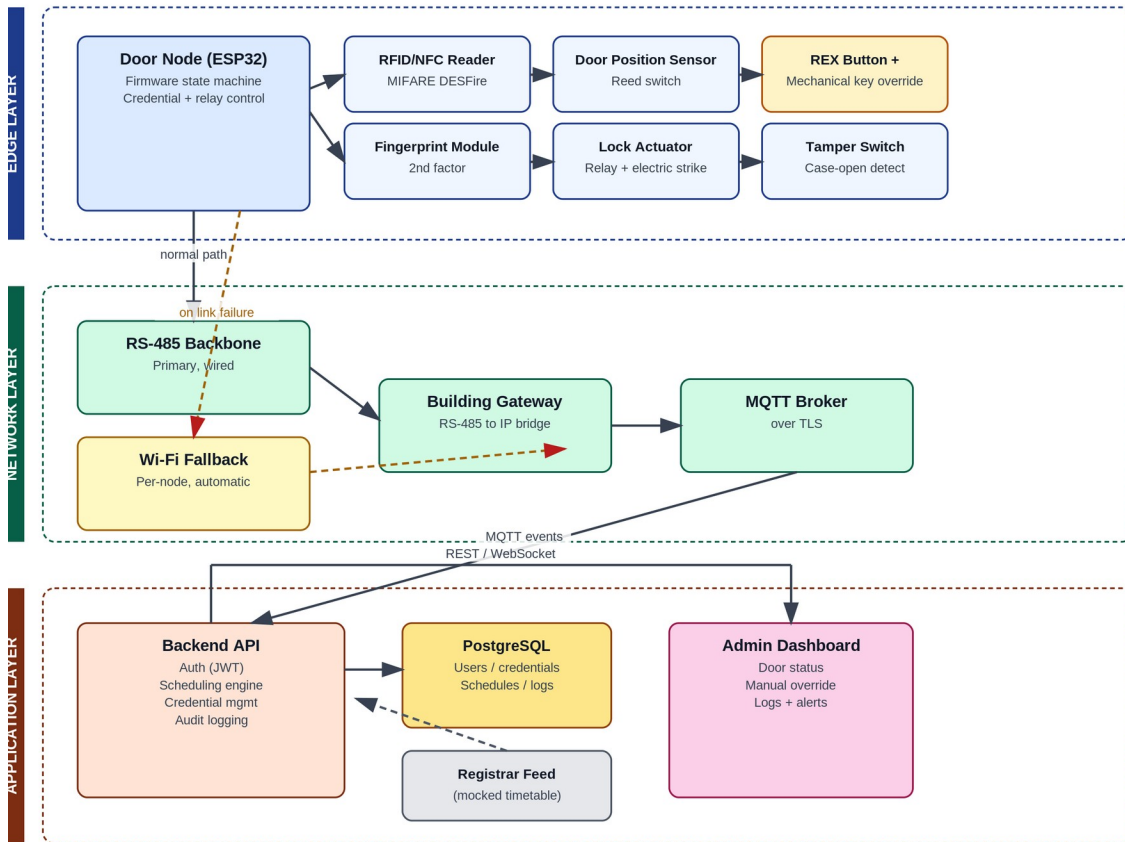


Figure 4.1 — Three-layer system architecture.

4.1 Edge Layer

Each door node is an ESP32 microcontroller driving: a PN532 NFC/RFID reader (SPI), an R307 fingerprint module (UART) for doors requiring a second factor, a magnetic reed switch for door position, a request-to-exit (REX) button, a mechanical key override, an electric strike via a relay, and a tamper (case-open) switch on an interrupt-capable pin.

4.2 Network Layer

Door nodes talk to a Building Gateway over a shared RS-485 multi-drop bus, with direct Wi-Fi/MQTT as an automatic fallback if the bus or gateway becomes unreachable. The gateway polls each node in round-robin order and bridges whatever it hears to the same MQTT topics a directly-connected node would use, so the backend requires no changes to support either transport.

Phase 6 — RS-485 Multi-Drop Bus Topology

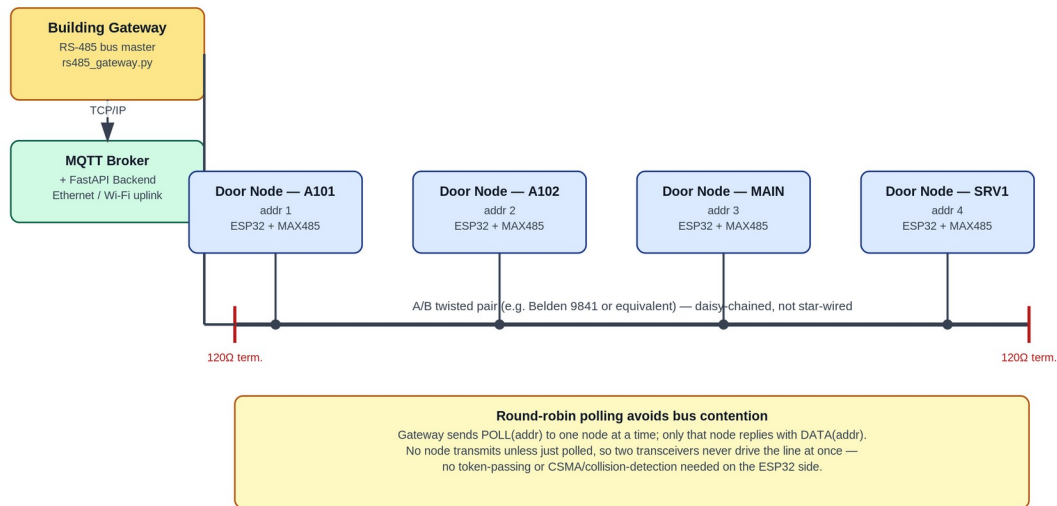


Figure 4.2 — RS-485 multi-drop bus topology: Building Gateway as bus master, door nodes as addressed slaves.

4.3 Application Layer

A FastAPI backend exposes a REST API (Appendix: API_Reference.docx) backed by SQLAlchemy (SQLite by default, PostgreSQL for production scale), ingests live MQTT status/events/alerts in a background thread, and enforces JWT authentication with role-based access control. A React (Vite) single-page dashboard polls the backend every 5 seconds to show live door status, alerts, and access logs, with remote override for admins.

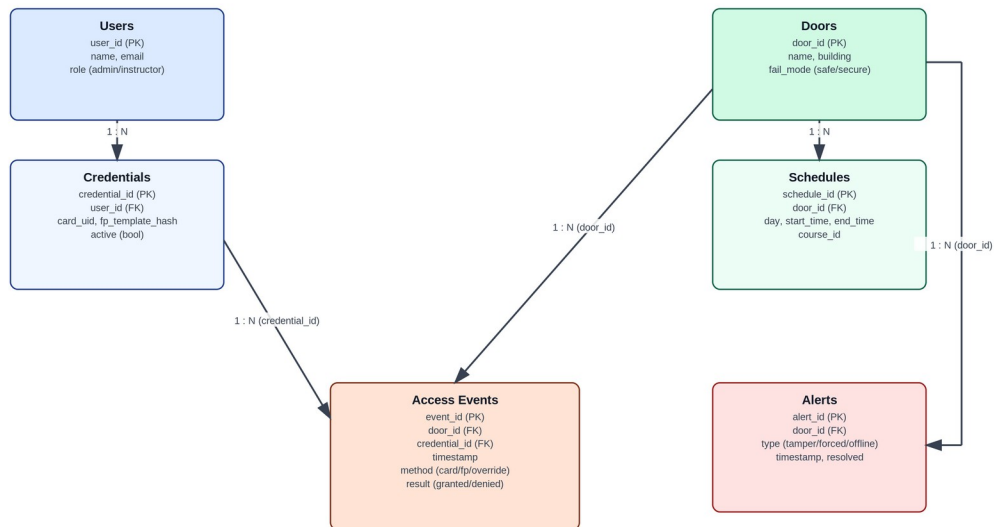


Figure 4.3 — Database schema (entity-relationship diagram).

5. Implementation

5.1 Firmware (Edge Layer)

Implemented as a FreeRTOS state machine (BOOT -> IDLE -> CARD_SCAN -> [BIOMETRIC_CHECK] -> ACCESS_DECISION -> GRANTED/DENIED -> MONITOR_DOOR -> RELOCK) on core 1, with a network watchdog task on core 0 handling both Wi-Fi/MQTT reachability and, since Phase 6, RS-485 poll reception. Key features implemented: MIFARE DESFire EV1 native AES-128 mutual authentication (challenge-response handshake via mbedTLS) so a card's identity is cryptographically proven rather than trusted from its UID alone; a tamper interrupt that forces an immediate relock and blocks further card-based entry until an admin clears it over MQTT, while leaving the REX button (local egress) always functional; and a RAM-based offline event buffer so access events survive a temporary broker outage instead of being dropped.

5.2 Building Gateway (Network Layer)

A Python service bridging the RS-485 bus to MQTT. It is the bus master in a from-scratch frame protocol (SOF/ADDR/TYPE/LEN/PAYLOAD/CHECKSUM/EOF with PPP-style byte stuffing), polling one node at a time so no two transceivers ever drive the shared line at once — avoiding bus contention without needing token-passing or collision detection on the ESP32 side.

5.3 Backend (Application Layer)

FastAPI + SQLAlchemy + Pydantic v2, with routers for auth, users, credentials, doors, schedules, and alerts (full surface in API_Reference.docx). Credential card UIDs are encrypted at rest with Fernet and looked up via a deterministic keyed-HMAC blind index, avoiding both plaintext storage and a full-table decrypt-and-compare on every lookup. Login attempts are rate-limited per account. A background MQTT thread ingests door status/events/alerts independently of the request/response cycle, so a broker outage doesn't take the REST API down with it.

5.4 Dashboard (Application Layer)

A React SPA (login, live door-status grid, alert banner, access-log table) polling the backend every 5 seconds. Overrides and alert resolution are restricted to the admin role at the UI layer and, more importantly, re-enforced server-side (the actual security boundary — see Section 6).

6. Security Design

Security was treated as an explicit, tracked requirement rather than an afterthought, with a dedicated review phase (Phase 5) producing a threat model, a before/after gap analysis, and a residual-risk register. Key mitigations implemented and tested:

Risk	Mitigation
Cloned/emulated card (UID-only reads)	DESFire EV1 AES-128 mutual authentication — a card must prove it holds the shared key, not just present a matching UID
Credential data exposed via DB access	Fernet encryption at rest + keyed-HMAC blind index for lookup, never storing or logging plaintext UIDs
Login brute-forcing	Per-account lockout after repeated failures (tested live: 429 after 5 failed attempts)
Tamper event only alerted, not acted on	Forced immediate relock + entry lockout until admin-cleared; REX/egress unaffected
UI-only access control (bypassable)	Role checks enforced server-side (require_admin dependency), not just hidden in the dashboard's UI

Residual risks, documented rather than hidden: all cards at a site currently share one AES key (no per-card key diversification); the DESFire handshake and RS-485 firmware logic are reviewed against their specifications but not run on real hardware; JWT sessions have no revocation list before expiry; and rate limiting is per-process (would need a shared store for a multi-instance backend deployment). Full detail in Phase5_Security_Review.docx.

7. Testing & Verification

Testing was layered: unit/integration tests per component, then multi-node load testing, then full-system fault injection — each phase building on real, executed evidence rather than assumed correctness.

Layer	Method	Result
Backend	pytest (auth, RBAC, doors, credentials, schedules, alerts, crypto, MQTT ingestion, staleness watchdog) + live curl/HTTP against a running server	43 / 43 passing
RS-485 wire protocol	Independent C (gcc) and Python implementations cross-checked byte-for-byte	5 / 5 passing
Dashboard	npm production build + scripted integration replay of its exact API call sequence against a live backend	Build clean, integration verified
Multi-node scaling	Concurrent load test: 12 simulated nodes (3 Hz) + 4 gateway-relayed nodes + 15 concurrent dashboard clients against a live backend	288/288 events ingested (0 drops), 4,168 REST requests at 479 req/s (0 errors)
Full-system scenario	Scripted admin session (login, door status, tamper alert, override, rate limiting) against the complete live stack	12 / 12 checks passed
Fault injection	Building Gateway process killed mid-run to simulate a crash	Found 1 defect (Section 7.1), fixed and re-verified live

7.1 Defect Found and Fixed

Fault injection revealed that a door's online status never recovered if the Building Gateway relaying it crashed outright (no clean MQTT disconnect to trigger the existing offline-detection paths). Fixed with a transport-agnostic staleness watchdog that periodically flags any door whose last-seen timestamp has gone stale, regardless of how it connects. Verified with 4 new unit tests and a live re-run of the same fault injection, confirming all affected doors were correctly marked offline within seconds. Full write-up in Phase7_System_Test_Report.docx.

7.2 What Remains Unverified

Stated plainly, consistent with this project's approach throughout: the ESP32 firmware (DESFire handshake, RS-485 primary/fallback logic) has never been compiled or run on real hardware in this development environment, and no real RS-485 electrical bus (wiring, termination, real transceivers) was available to test against — only a virtual serial link. These are the concrete next steps for anyone continuing this project, not gaps papered over in this report.

8. Results & Evaluation

Against the objectives in Section 1.2: credential-based access with cryptographic authentication, tamper-active response, and encrypted storage were all implemented and tested (to the extent possible without real hardware). The backend and dashboard were demonstrated working together live, including under concurrent multi-node load with zero data loss at pilot scale. The hybrid RS-485/Wi-Fi backbone and Building Gateway — the piece of the original architecture diagram that took longest to become real — is implemented, its wire protocol formally cross-verified, and its integration with the rest of the stack proven via a live gateway-crash fault injection that both found and fixed a real defect.

The clearest limitation against the original ambition is hardware verification: this project was developed in an environment without ESP32 hardware or a physical RS-485 bus, so firmware-level claims rest on rigorous code review and (for the RS-485 wire format specifically) independent cross-language verification, rather than a bench test. This is reported as an open item, not resolved by assertion.

9. Project Management

Delivered across 10 phases over a 34-week plan: Proposal & Planning, System Design, Single Door-Node Prototype, Backend & Database, Dashboard, Security & Fail-Safe, Multi-Node Scaling, Full System Testing, Documentation, and Presentation/Defense Prep. Estimated pilot hardware budget: \$635 for a 5-door deployment (Access_Control_System_Proposal_Revised.docx, Section 11). Full task-level timeline in Access_Control_System_Project_Timeline.xlsx.

10. Conclusion & Future Work

This project delivers a working, layered access control system — firmware through backend through dashboard through a multi-node communication backbone — with security treated as a tracked, tested requirement rather than an afterthought, and with an unusually high proportion of its claims backed by real, executed tests rather than description alone. The most valuable next steps for continuing this work are, in priority order: bench-test the firmware (DESFire handshake and RS-485 logic) on real ESP32 hardware against real cards and a real bus; implement per-card AES key diversification to remove the single-shared-key risk; add JWT session revocation; and build dedicated dashboard screens for credential/schedule management, currently API-only (User_Manual.docx, Section 6).

11. References

- [1] NXP Semiconductors, MIFARE DESFire EV1 native command set and AES authentication protocol documentation.
- [2] Espressif Systems, ESP32 Technical Reference Manual (FreeRTOS dual-core task model, HardwareSerial/UART peripherals).
- [3] Eclipse Foundation, Mosquitto / paho-mqtt documentation (MQTT 3.1.1 protocol).
- [4] FastAPI, SQLAlchemy, and Pydantic official documentation.
- [5] IETF RFC 1662, PPP in HDLC-like Framing (byte-stuffing technique adapted for the RS-485 frame protocol, Section 5.2).

Appendices

The following standalone documents, produced during this project, are included as appendices and referenced throughout this report rather than duplicated in full:

- Access_Control_System_Proposal_Revised.docx — original project proposal
- System_Design_Document.docx — full architecture, schema, and API/MQTT specification
- Phase2_Wiring_and_Bench_Test_Guide.docx — single-node hardware assembly and bench test procedure
- Phase5_Security_Review.docx — threat model, gap analysis, residual risks
- Phase6_Multi_Node_Deployment_Guide.docx — RS-485 wiring/addressing and multi-node load test results
- Phase7_System_Test_Report.docx — full-system test results and the staleness watchdog defect
- API_Reference.docx — complete REST/MQTT interface reference
- Installation_Deployment_Guide.docx — consolidated setup instructions for every component
- User_Manual.docx — end-user guide for the dashboard
- Access_Control_System_Project_Timeline.xlsx — full task-level project schedule